PROJECT OVERVIEW

Efficient Text Recognition

Reading printed and handwritten text from images with one CNN and three interchangeable “readers”: BiLSTM, Transformer and Mamba. All three are trained from scratch on the same data and compared on the same test lines.

CNN + BiLSTM

CNN + Transformer

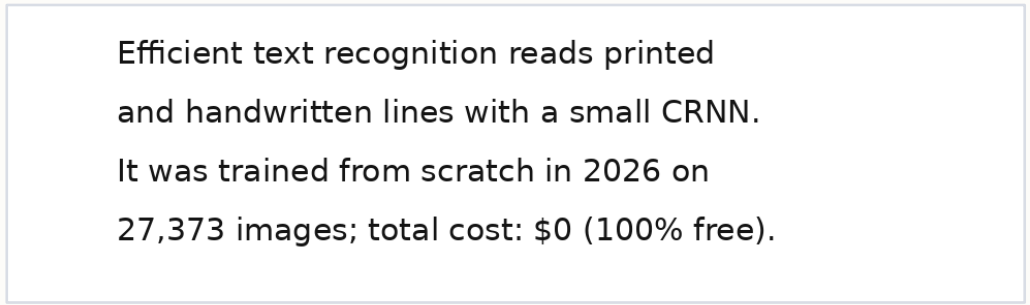
CNN + Mamba

Best result: CNN + Mamba, 7.4% wrong characters on handwriting, 1.6% on print

THE PROBLEM

Turn a picture of text into text you can edit

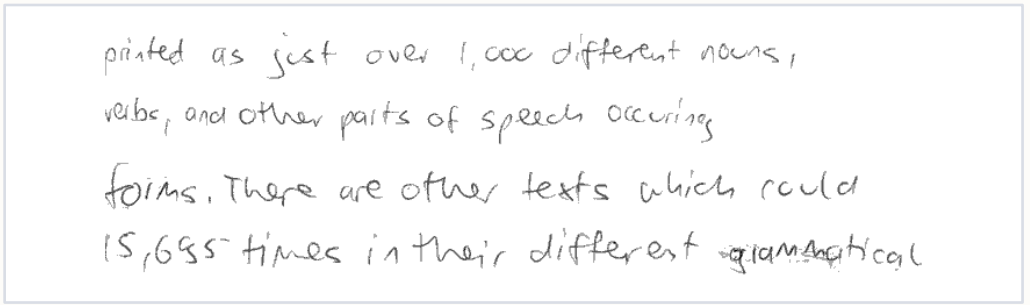
PRINTED · IN



TEXT · OUT

```
Efficient text recognition reads printed
and handwritten lines with a small CRNN.
It was trained from scratch in 2026 on
27.373 images; total cost: $0 (100% free) .
```

HANDWRITTEN · IN



TEXT · OUT

```
pinted as just over l ,oo diftereut nous ,
veibs , and other parts of speech occuring
foims , There are other texts which rould
(S,66s-timec in their different ygramsatical
```

Real output of the original model (CNN + BiLSTM). Printed text is nearly perfect; handwriting is readable but has about one wrong character in eleven.

THE CLIENT BRIEF

The CNN stays. Only the reader after it changes.

HOW THE BRIEF READS

Try one method, switch if it fails

Start with a CNN. If it does not work well enough, use a Transformer instead. Then try Mamba or diffusion.

This suggests each method replaces the one before.

HOW OCR ACTUALLY WORKS

Same eyes, three different readers

Every modern OCR model starts with a CNN that looks at the pixels. A sequence model then reads its output left to right: a BiLSTM, a Transformer or Mamba.

We build all three on one shared CNN and measure them on the same test lines. Diffusion creates images and does not read them, so it fits at the data step (slide 14).

HOW IT WORKS

Six steps from picture to text

1

Clean

Even out the paper, make text dark

2

Find lines

Cut the page into text lines

3

Resize

Every line becomes 40 px tall

4

CNN

Pixels into one feature vector per column

5

Reader

BiLSTM, Transformer or Mamba reads in order

6

CTC

Column guesses into the final text

Steps 1–3: fixed image processing, no learning

Steps 4–6: the neural network, learned from data

Only step 5 differs between the three models. Everything else is shared, so the comparison is fair.

STEP 1 · CLEAN

Make every photo look like dark ink on white paper

1. Photo as uploaded (light text on a dark, uneven background)



2. After clean-up: dark text on even white paper



1 **Flip dark themes**

If most of the image is dark, the colours are inverted so the text becomes the dark part.

2 **Remove shadows**

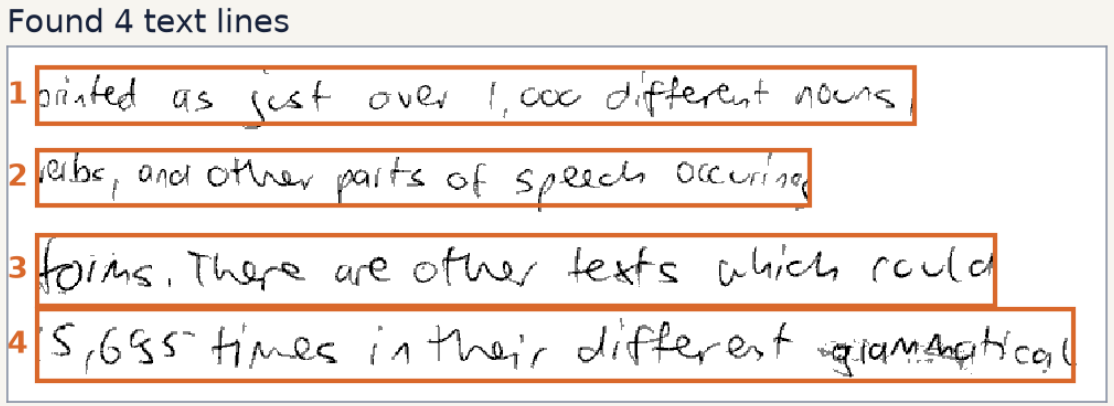
A blurred maximum filter estimates the paper brightness around every pixel; dividing by it evens out shadows.

3 **Stretch the contrast**

The darkest ink is pushed to black, so faint pencil and dark marker look alike.

STEP 2 · FIND LINES

Cut the page into lines by counting ink per row





1 Ink profile

Otsu's method picks a threshold between ink and paper; then the ink pixels in each row are counted (the blue chart).

2 Peaks are lines

Runs of busy rows become lines. Thin runs such as dots and accents are merged into the line next to them.

3 Room for tails

Each box grows a little so g, y and p keep their tails, then it is trimmed left and right to the ink.

STEP 3 · RESIZE

Every line becomes 40 pixels tall, whatever its width

Cropped line: 525 x 42 px



Resized to 40 px high (614 x 40), grey 0-1



1 **Fixed height**

The network expects 40-pixel lines, so each line is scaled to 36 px plus a 2 px white border.

2 **Any width**

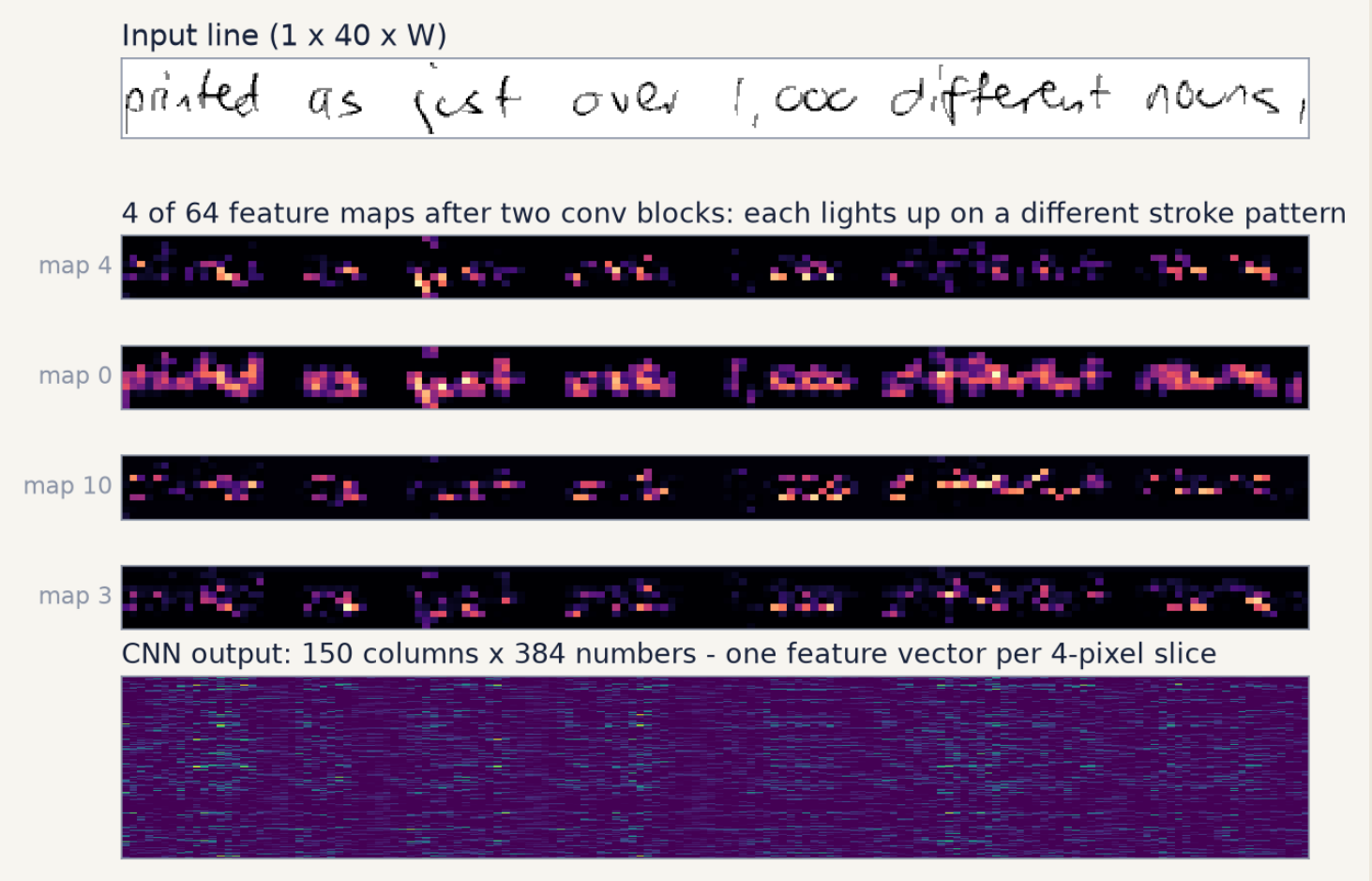
The aspect ratio is kept, so a long line simply gives the model more columns to read.

3 **Numbers, not pixels**

Grey levels become values between 0 and 1, with ink as the high value.

STEP 4 · CNN, THE EYES

The CNN turns pixels into one feature vector per column



1 **5 convolution layers**

Small 3×3 filters (32 up to 192 of them) learn to respond to edges, curves, loops and stroke ends.

2 **Pooling shrinks it**

Height goes 40 → 2 and width shrinks by 4. Each 4-pixel slice of the line becomes one column.

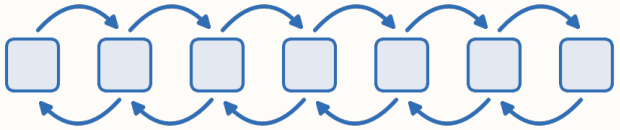
3 **384 numbers per column**

192 filters × 2 rows describe what is drawn in that slice. This sequence goes to the reader.

STEP 5 · THE READER: THE ONLY PART THAT CHANGES

Three ways to read the columns in order

BiLSTM

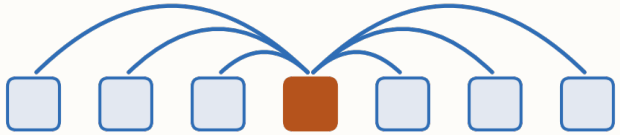


How: reads one column at a time, left to right and right to left, carrying a memory.

Good: small, learns well from little data; the classic OCR design since 2015.

Weak: steps run one after another, so it trains less in parallel.

Transformer

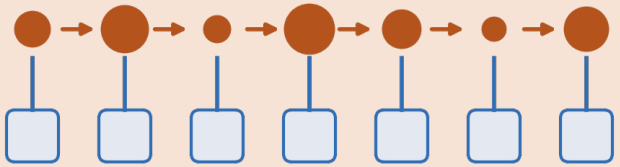


How: every column looks at every other column at once (self-attention).

Good: fully parallel; the strongest design when pre-trained on huge data.

Weak: needs a lot of data, and its cost grows with length squared.

Mamba



How: reads in order with a memory that chooses, per column, what to keep or forget.

Good: cost grows only linearly with length; trains in parallel.

Weak: new (2023–24) and less proven for OCR than the other two.

INSIDE MAMBA

A memory that decides what to keep, column by column

```
h[t] = a[t]·h[t-1] + b[t]·x[t]
y[t] = c[t]·h[t]
```

h is the memory, **x** is the column from the CNN and **y** is what goes on to CTC.

Selective: a, b and c are computed from x itself. A space or a clear letter can wipe or keep the memory as needed. This is what Mamba adds to older state-space models.

57 s → 0.18 s

per training step. The first version stepped through the columns in a Python loop; the final one computes the same recurrence as one matrix product (Mamba-2's "SSD" form).

4.5 → 1.3 GB

GPU memory, so it fits a 4 GB laptop GPU. A unit test proves both versions give the same output.

STEP 6 · CTC

One guess per column, then merge and drop blanks

The model gives one guess per 4-pixel column (38 columns)



merge repeats, drop blanks (·) → “Invoice”

1 **84 choices per column**

83 characters plus a special “blank” meaning “no new letter here”. Most columns are blank.

2 **Decode**

Merge repeated guesses (“i i” → “i”), then remove blanks. A blank between two l’s keeps the double l in “hello”.

3 **Train without positions**

The CTC loss adds up every alignment that spells the right text, so nobody marks where each letter is.

TRAINING

Same data, same recipe, same test lines for all three

10,373

real handwritten lines by about 650 writers (IAM)

17,000

printed words in 170 fonts, joined into random lines

240,000

lines seen per model: 30 epochs × 250 batches × 32

~20 min

per model on a laptop RTX 3050 GPU (hours on a CPU)

Augmentation

Every line is randomly slanted, stretched, blurred, thickened or thinned, faded and given noise, so the model never sees the same image twice.

Optimiser

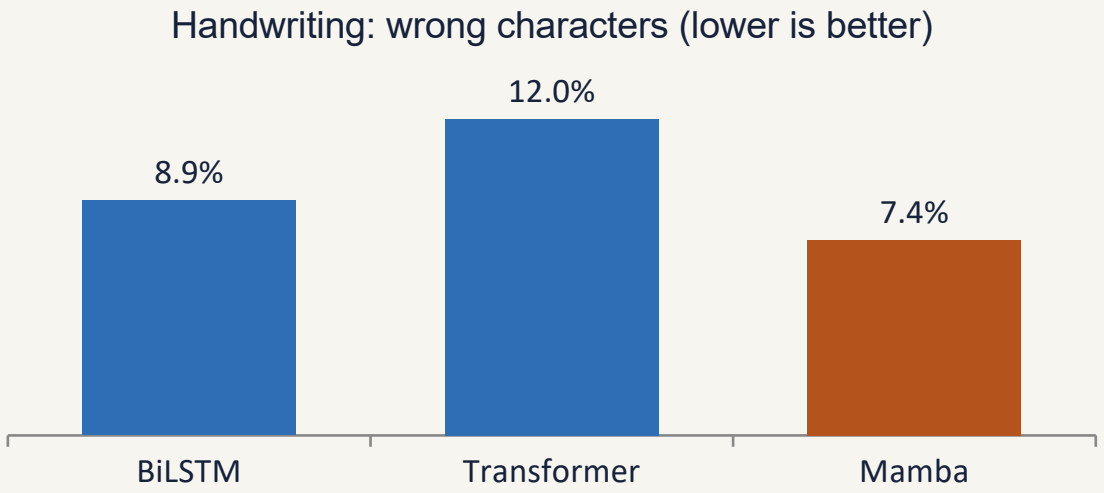
AdamW with a one-cycle learning rate (peak 0.001), gradient clipping and mixed precision on the GPU.

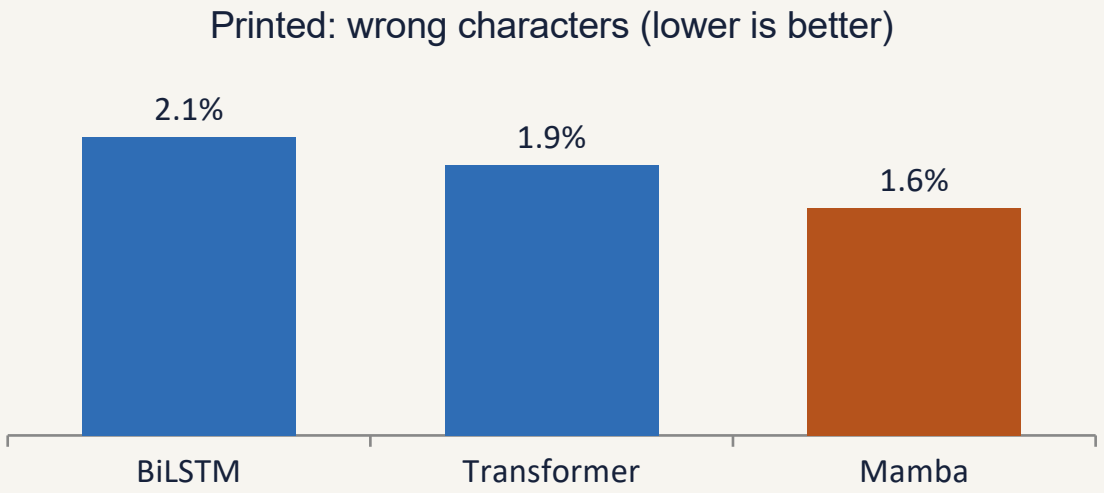
Fair test

After every epoch: 960 held-out IAM lines and 1,920 printed lines. The best epoch is kept. Test lines are never trained on.

RESULTS

Mamba reads best, on handwriting and on print





Model	Size	Handwriting wrong	Handwritten lines perfect	Printed lines perfect	Speed, ms per line
CNN + BiLSTM	2.21 M	8.9%	15.9%	80.1%	3.1
CNN + Transformer	2.28 M	12.0%	6.4%	83.1%	0.9 (fastest)
CNN + Mamba (best)	**1.89 M**	**7.4%**	**20.1%**	**88.1%**	**1.6**

WHERE DIFFUSION FITS

Diffusion models draw images. They do not read them.

USE 1 · MORE TRAINING DATA

Generate new handwriting

A diffusion model trained on IAM can write any sentence in many styles. That gives the reader thousands of extra labelled lines, the main fix for weak handwriting accuracy.

USE 2 · BETTER INPUT

Restore bad scans

Before step 1, a diffusion model can sharpen blurry photos or remove stains and fold marks, so the reader gets cleaner lines.

Recommendation: keep diffusion as a phase 2 data and restoration step. It is slow (dozens of network passes per image) and would not replace the reader.

USING IT

One command: pick a model, train it, compare, read

```
> python run.py

  1  CNN + BiLSTM         handwriting 8.9%,  printed 2.1%
  2  CNN + Transformer    handwriting 12.0%, printed 1.9%
  3  CNN + Mamba          handwriting 7.4%,  printed 1.6%

  1-3  open the web app with that model
  t    train a model from scratch
  c    compare all trained models
  q    quit

Your choice [3]:
```

1 **First run sets itself up**

Installs PyTorch, NumPy and Pillow if missing and unpacks the 28 MB dataset.

2 **Web app**

Upload, paste or drop an image, or write on the drawing pad. Each line is shown with its confidence.

3 **Scripted use**

--arch, --train, --compare and --image do the same without the menu.

RECOMMENDATION

Use CNN + Mamba, then spend effort on handwriting data

NOW

Ship Mamba as default

Most accurate on both kinds of text and the smallest model. It is already the default choice in run.py.

NEXT

Train longer, add a word model

Mamba was still improving at epoch 30. A small language model over the CTC output would fix near-misses like “pinted”.

PHASE 2

More handwriting

Handwriting is the weak spot for every reader. More real lines, or diffusion-generated ones, help more than another architecture.

Open questions for the client: target error rate, handwriting or print priority, and GPU or CPU deployment.